# SVESVARAYA TECHNOLOGICAL UNIVERSITY

**"JnanaSangama", Belgaum -590014, Karnataka.**

**DATA STRUCTURES AAT REPORT**
**on**

# DATA STRUCTURES (23CS3PCDST)

***Submitted by***

**Sagarmatha Khatri (1BF24CS261)**

***\in partial fulfillment for the award of the degree of***
**BACHELOR OF ENGINEERING**
***in***
**COMPUTER SCIENCE AND ENGINEERING**

**B.M.S. COLLEGE OF ENGINEERING**
**(Autonomous Institution under VTU)**
**BENGALURU-560019**
**September 2025 to January 2026**

**B. M. S. College of Engineering,**
**Bull Temple Road, Bangalore 560019**

**(Affiliated To Visvesvaraya Technological University, Belgaum)**
**Department of Computer Science and Engineering**


This is to certify that the Lab work entitled "**DATA STRUCTURES**" carried out by SAGARMATHA KHATRI (1BF24CS261), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2025-2026. The Lab report has been approved as it satisfies the academic requirements in respect of Data structures Lab - **(23CS3PCDST)** work prescribed for the said degree.

**Dr. Anusha S**
Assistant Professor
Department of CSE
BMSCE, Bengaluru

**Dr. Kavitha Sooda**
Professor and Head
Department of CSE
BMSCE, Bengaluru

# INDEX

**Course Outcomes:**

| | |
|---|---|
| CO1 | Apply the concept of linear and nonlinear data structures. |
| CO2 | Analyze data structure operations for a given problem |
| CO3 | Design and develop solutions using the operations of linear and nonlinear data structure for a given specification. |
| CO4 | Conduct practical experiments for demonstrating the operations of different data structures. |

# LAB program-1

**Write a program to simulate the working of stack using an array with the following:**
**a) Push**
**b) Pop**
**c) Display**
**The program should print appropriate messages for stack overflow, stack underflow**

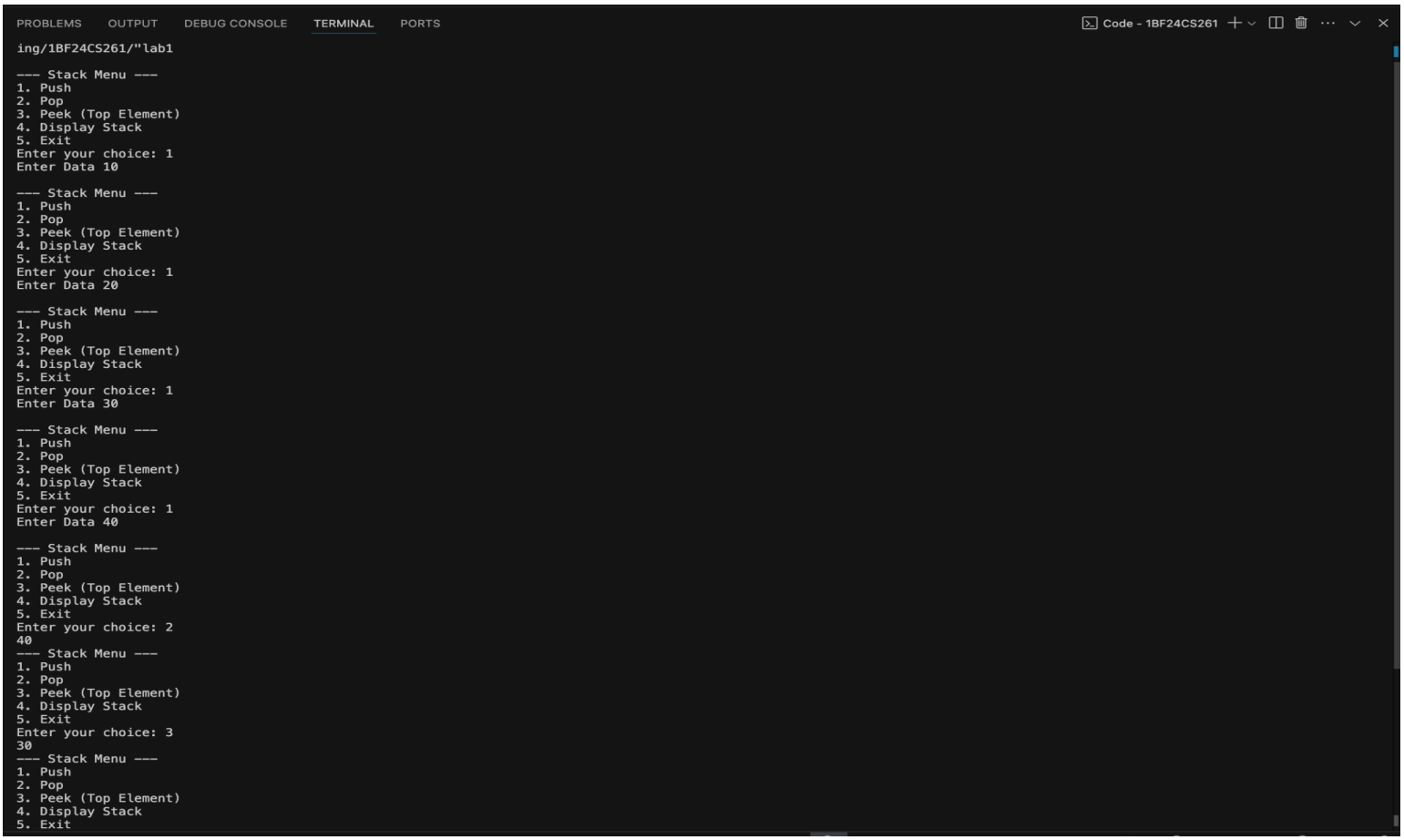
//CODE:

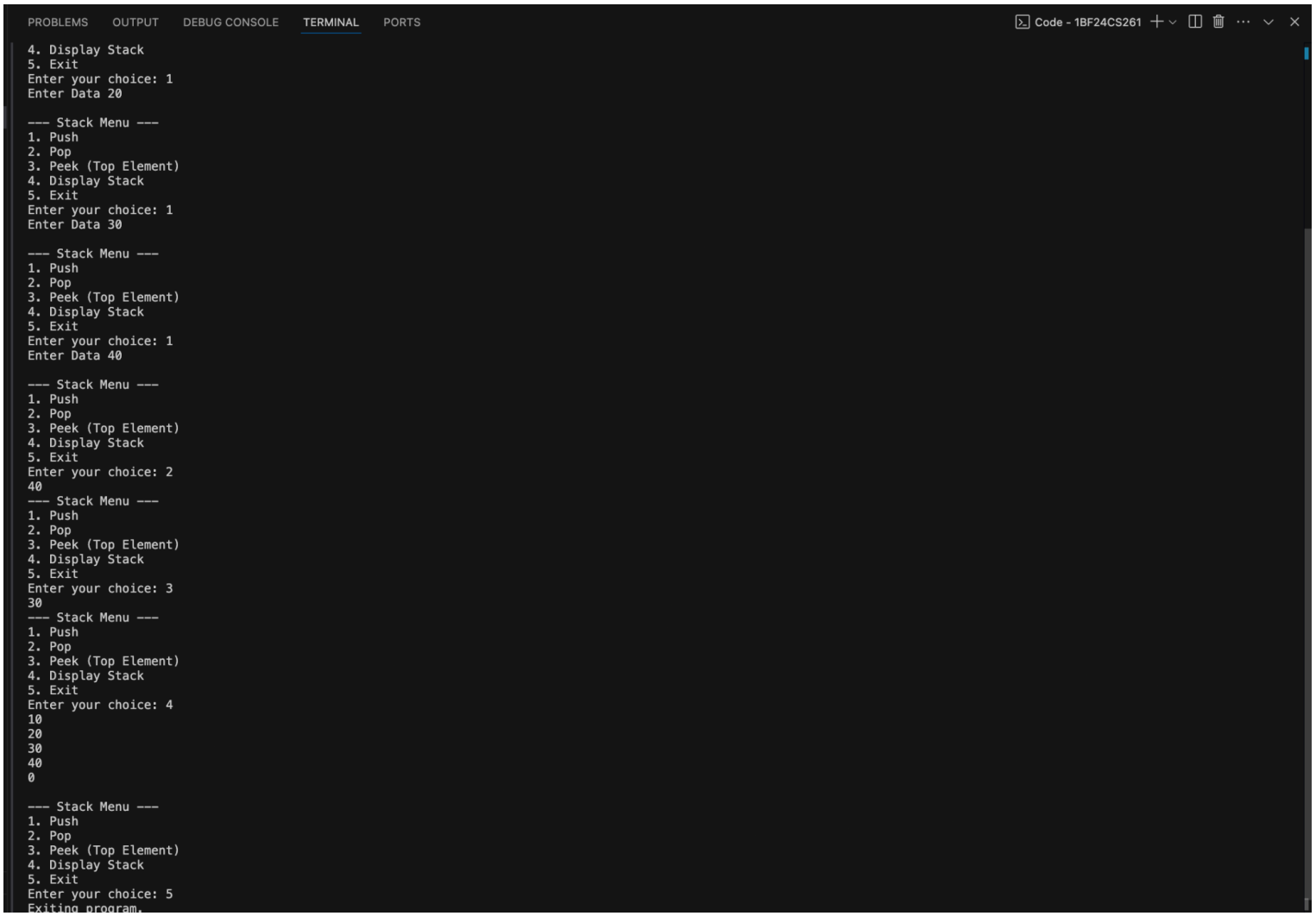
```
#include<stdio.h>
#define n
int stack[n];
int top=-1;
void push()
{
   int x;
   if (top==n-1){
      printf("STACKOVERFLOW \n");
   }else
   {
      printf("enter data:");
      scanf("%d", &x);
      stack[++top]=x;
      printf("%d pushed onto the stack \n");
   }
}

void pop()
{
   if (top==-1){
      printf("STACKUNDERFLOW");
   }else
   {
      printf("\n pop from the stack= %d", stack[top]);
      top--;
   }
}
```

```
void peek ()
{
   if (top==-1)
   {
      printf("stack is empty! \n");
   }else
   {
      printf("top element is %d \n", stack[top]);
   }
}

void display ()
{
   if (top==-1)
   {
      printf("stack is empty\n");
   }else
   {
      printf("stack elements are: \n");
      for (int i= top; i>=0;i--){
         printf("%d\n, stack[i]");
      }
   }
}


int main()
{
   int choice;
   while (1){
      printf ("\n stack");
      printf ("1.push \n");
      printf ("2.pop \n");
      printf ("3.peek \n");
      printf ("4.exit\n");
      printf ("5.display\n");
      printf ("enter your choice");
      scanf("%d",&choice);
      switch(choice)
      {
      case 1:
         push();
         break;
```

```
        case 2:
            pop();
            break;
        case 3:
            peek();
            break;
        case 4:
            printf("exiting the code bye bye\n");
            return 0;
        case 5:
            display();
            break;
        default:
            printf ("invalid choice!!!\n");
            break;
        }
    }
    return 0;
}
```

**OUTPUT:**




PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code - 1BF24CS261
4. Display Stack
5. Exit
Enter your choice: 1
Enter Data 20
--- Stack Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice: 1
Enter Data 30
--- Stack Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice: 1
Enter Data 40
--- Stack Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice: 2
40
--- Stack Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice: 3
30
--- Stack Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice: 4
10
20
30
40
0
--- Stack Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice: 5
Exiting program.

# LAB program-2

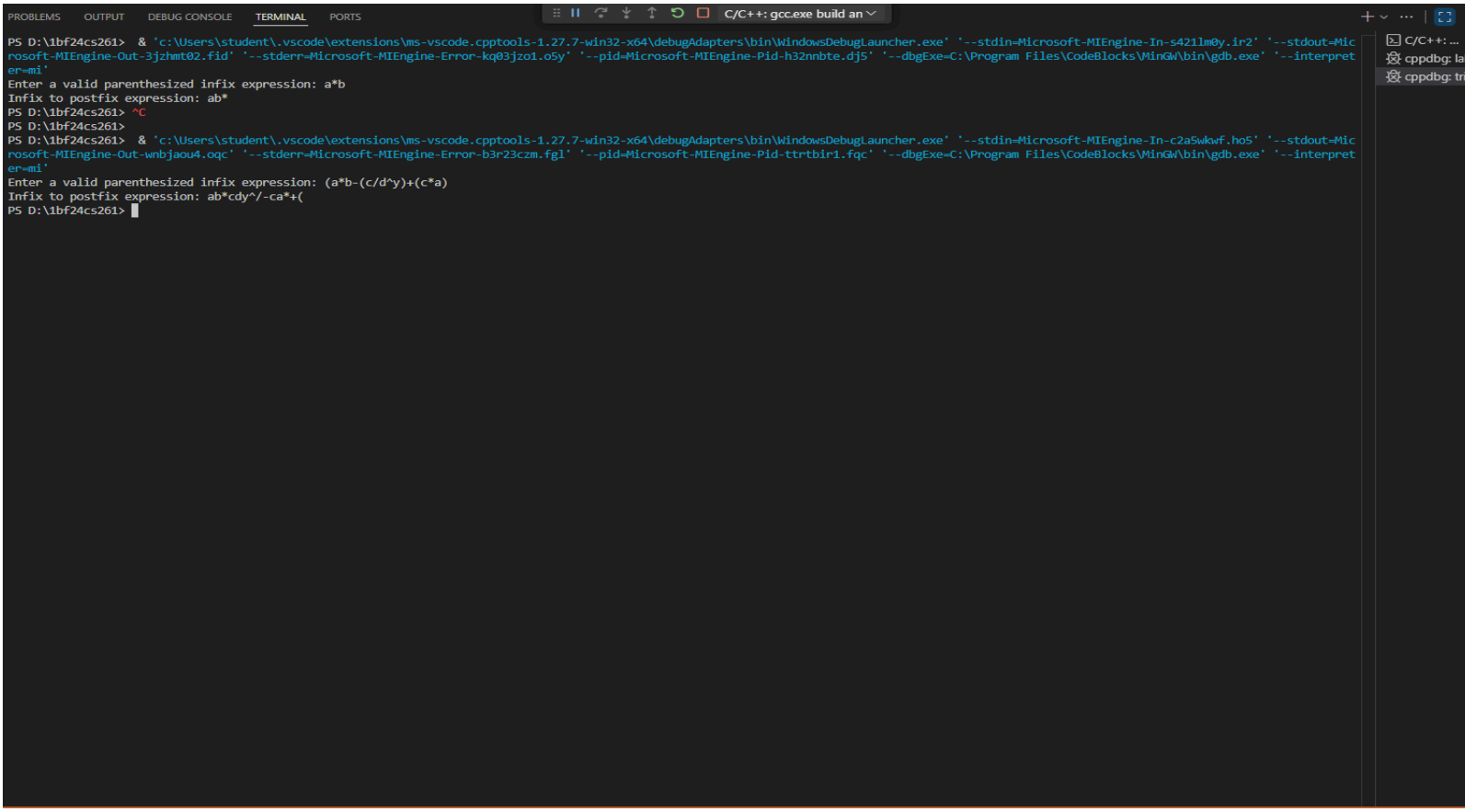
**(a) WAP to convert a given valid parenthesized infix arithmetic expression to postfix expression. The expression consists of single character operands and the binary operators + (plus), - (minus), * (multiply) and / (divide)**

//CODE:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<ctype.h>

#define MAX 100
int top = -1;
int stack[MAX];

void push(char c) {
    if (top == MAX - 1) {
        printf("STACKOVERFLOW\n");
        return;
    }
    stack[++top] = c;
}

char pop() {
    if (top == -1) {
        printf("STACKUNDERFLOW\n");
        return -1;
    }
    return stack[top--];
}

char peek() {
    if (top == -1) return -1;
    return stack[top];
}
```

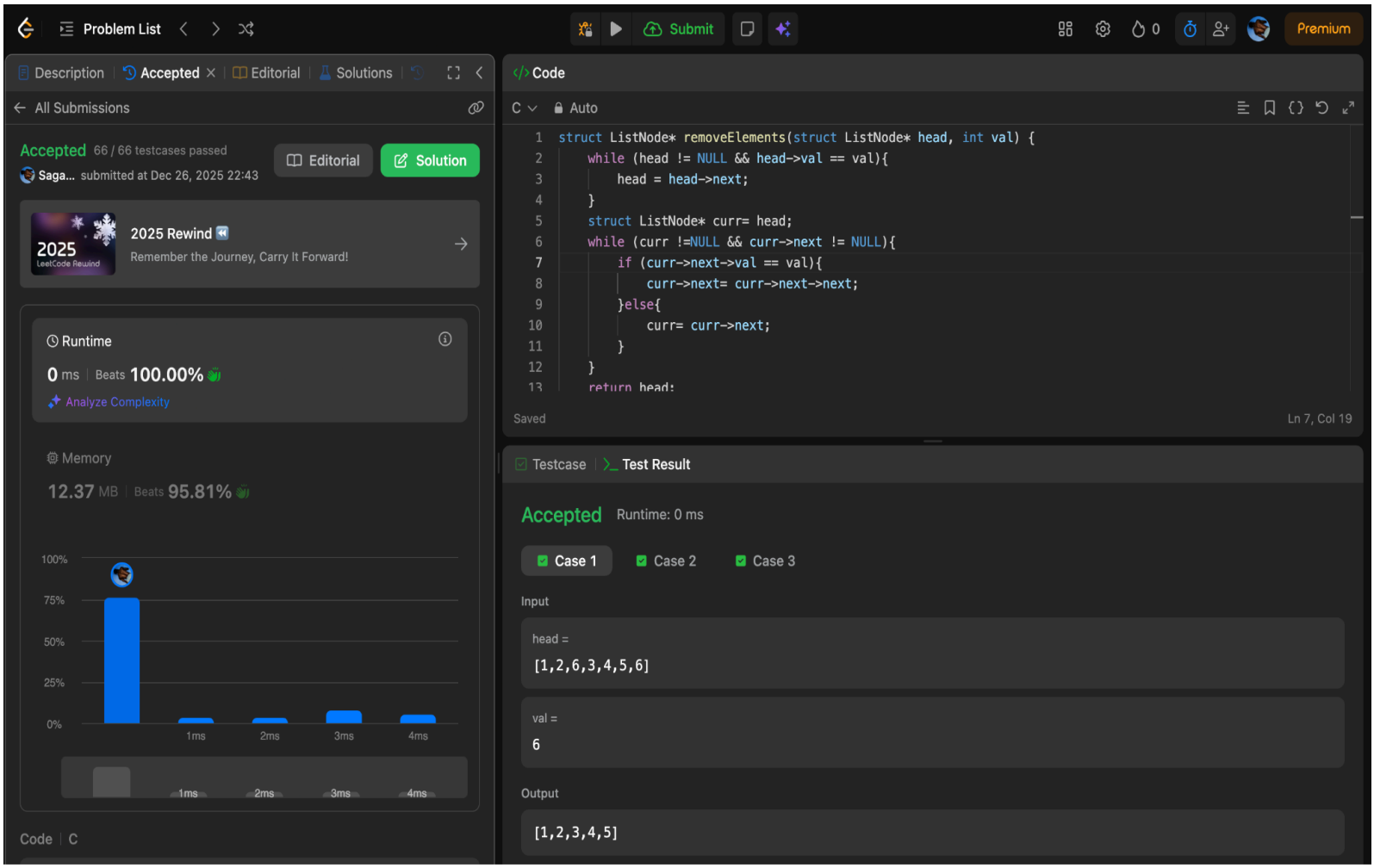
```
int precedence(char op) {
    switch (op) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
        case '(':
            return 0;
    }
    return -1;
}

int associativity(char op) {
    if (op == '^') {
        return 1;
    }
    return 0;
}

void infixToPostfix(char infix[], char postfix[]) {
    int i, k = 0;
    char c;
    for (i = 0; infix[i] != '\0'; i++) {
        c = infix[i];
        if (isalnum(c)) {
            postfix[k++] = c;
        } else if (c == '(') {
            push(c);
        } else if (c == ')') {
            while (peek() != '(') {
                postfix[k++] = pop();
            }
            pop();
        } else {
            while (top != -1 &&
                   ((precedence(peek()) > precedence(c)) ||
```

```
            (precedence(peek()) == precedence(c) && associativity(c) == 0))) {
            postfix[k++] = pop();
        }
        push(c);
    }
}

while (top != -1) {
    postfix[k++] = pop();
}
postfix[k] = '\0';
}

int main() {
    char infix[MAX], postfix[MAX];
    printf("Enter a valid parenthesized infix expression: ");
    scanf("%s", infix);
    infixToPostfix(infix, postfix);
    printf("Infix to postfix expression: %s\n", postfix);
    return 0;
}
```



(b) LEETCODE 1 :

**You are given a singly linked list where you are given the head of the list and an integer value `val`. Your task is to remove all nodes whose data/value is equal to `val` from the linked list. While deleting, you must correctly handle cases where the node to be removed is the head node, a middle node, or the last node. After removing all such nodes, you should return the updated head of the linked list.**



# LAB program-3

**(a) WAP to simulate the working of a queue of integers using an array. Provide the following operations: Insert, Delete, Display The program should print appropriate messages for queue empty and queue overflow conditions**

//CODE:

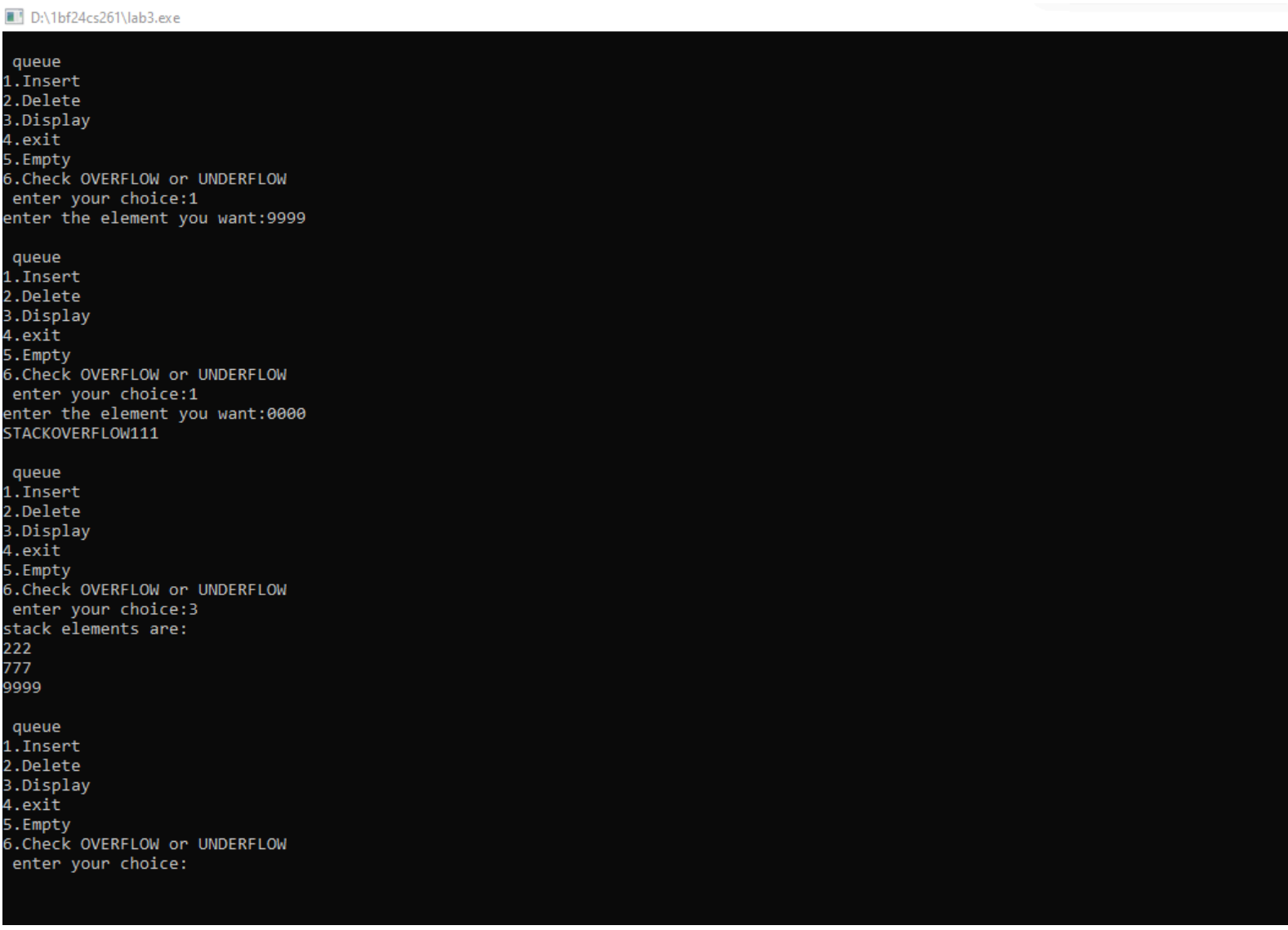
```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define N 3
int queue [N];
int front=-1;
int rear=-1;



void enque(int x)
{
  if (rear==N-1){
       printf("STACKOVERFLOW111 \n");
  }else if(front==-1 && rear==-1)
  {
     front++;rear++;
     queue[rear]=x;
  }else
  {
     rear++;
     queue[rear]=x;
  }
}



void deque()
{
   if(front==-1 && rear==-1)
```

```
    {
        printf("empty que");
    }else if (rear==front)
    {
        printf("the deleted element is %d\n",queue[front]);
        front=rear=-1;
    }else
    {
        int x=queue[front];
        front++;
        printf("the deleted element is %d\n",x);
    }
}



void display ()
{
    if (front==-1&&rear==-1)
    {
        printf("stack is empty\n");
    }else
    {
        printf("stack elements are: \n");
        for (int i= front; i<=rear;i++){
            printf("%d\n", queue[i]);
        }
    }
}


void ooru()
{
    if (front==-1 && rear==-1){
        printf("UNDERFlOW");
    }else if(rear>=N)
    {
        printf("OVERFLOW");
    }else
    {
```

```
        printf("elements are present!!!!");
    }
}


int main()
{
    int choice,x;
    while (1){
        printf ("\n queue\n");
        printf ("1.Insert \n");
        printf ("2.Delete \n");
        printf ("3.Display \n");
        printf ("4.exit\n");
        printf ("5.Empty\n");
        printf ("6.Check OVERFLOW or UNDERFLOW ");
        printf ("\n enter your choice:");
        scanf("%d",&choice);
        switch(choice)
        {
        case 1:
            printf("enter the element you want:");
            scanf("%d",&x);
            enque(x);
            break;
        case 2:
            deque();
            break;
        case 3:
            display();
            break;
        case 4:
            printf("exiting the code bye bye\n");
            return 0;
        case 5:
            display();
            break;
        case 6:
            ooru();
            break;
```

```
            default:
                printf ("invalid choice!!!\n");
                break;
            }
        }
        return 0;
    }
```

## OUTPUT:



**(b) WAP to simulate the working of a circular queue of integers using an array. Provide the following operations: Insert, Delete & Display The program should print appropriate messages for queue empty and queue overflow conditions**

//CODE:

```
#include <stdio.h>
#define MAX 5
int queue[MAX];
int front = -1;
int rear = -1;

void enqueue(int item)
{
   if (front == -1 && rear == -1)
   {
      front = 0;
      rear = 0;
      queue[rear] = item;
   }
   else if ((rear + 1) % MAX == front)
   {
      printf("Queue Overflow\n");
   }
   else
   {
      rear = (rear + 1) % MAX;
      queue[rear] = item;
   }
}

void dequeue()
{
   if (front == -1 && rear == -1)
   {
      printf("Queue Underflow!!\n");
   }
   else if (front == rear)
```

```
    {
        front = rear = -1;
    }
    else
    {
        printf("%d removed!!\n", queue[front]);
        front = (front + 1) % MAX;
    }
}

void display()
{
    if (front == -1 && rear == -1)
    {
        printf("Queue is empty!!\n");
    }
    else
    {
        for (int i = front; i <= rear; i = (i + 1) % MAX)
        {
            printf("%d\n", queue[i]);
        }
    }
}

int main()
{
    int value;
    int n;
    while (1)
    {
        printf("CHOOSE:\n 1.INSERT\n 2.DELETE\n 3.DISPLAY\n 4.EXIT\n");
        scanf("%d", &value);
        switch (value)
        {
        case 1:
            printf("Enter the value you want to insert\n");
            scanf("%d", &n);
            enqueue(n);
            break;
```

```
        case 2:
            dequeue();
            break;
        case 3:
            display();
            break;
        case 4:
            return 0;

        default:
            break;
        }
    }
    return 0;
}
```

**OUTPUT:**

```
CHOOSE:
 1.INSERT
 2.DELETE
 3.DISPLAY
 4.EXIT
1
Enter the value you want to insert
50
CHOOSE:
 1.INSERT
 2.DELETE
 3.DISPLAY
 4.EXIT
3
20
30
40
50
CHOOSE:
 1.INSERT
 2.DELETE
 3.DISPLAY
 4.EXIT
2
20 removed!!
CHOOSE:
 1.INSERT
 2.DELETE
 3.DISPLAY
 4.EXIT
3
30
40
50
CHOOSE:
 1.INSERT
 2.DELETE
 3.DISPLAY
 4.EXIT
```

# LAB program-4

**WAP to Implement Singly Linked List with following operations:**
**a) Createalinkedlist.**
**b) Insertion of a node at first position, at any position and at end of list.**
**Display the contents of the linked list.**

//CODE:

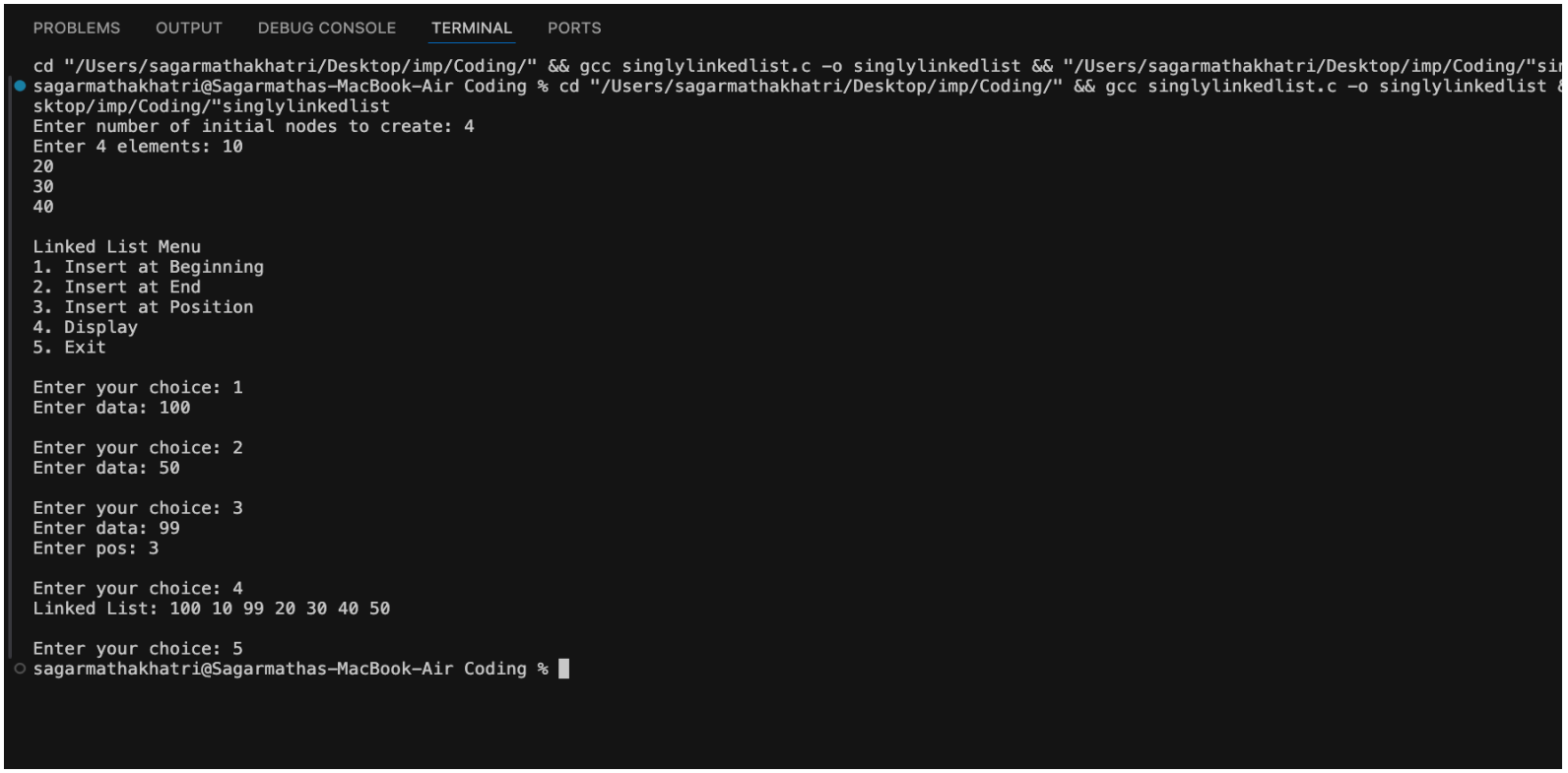
```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node* next;
};

struct Node* head = NULL;

void create(int n)
{
    struct Node *newNode, *temp;
    int data, i;
    if (n <= 0) return;
    for (i = 0; i < n; i++)
    {
        newNode = (struct Node*)malloc(sizeof(struct Node));
        scanf("%d", &data);
        newNode->data = data;
        newNode->next = NULL;
        if (head == NULL)
            head = newNode;
        else
        {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = newNode;
```

```
        }
    }
}

void insertAtBeginning(int data)
{
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = head;
    head = newNode;
}

void insertAtEnd(int data)
{
    struct Node *newNode, *temp;
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    if (head == NULL)
        head = newNode;
    else
    {
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
}

void insertAtPosition(int data, int pos)
{
    int i;
    struct Node *newNode, *temp;
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    if (pos == 1)
    {
        newNode->next = head;
        head = newNode;
        return;
```

```
    }
    temp = head;
    for (i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;
    if (temp == NULL) return;
    newNode->next = temp->next;
    temp->next = newNode;
}

void display()
{
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n, choice, data, pos;
    printf("Enter number of initial nodes to create: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    create(n);
    printf("\nLinked List Menu\n1. Insert at Beginning\n2. Insert at End\n3. Insert at
Position\n4. Display\n5. Exit\n");

    while (1) {
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);
                insertAtBeginning(data);
                break;
            case 2:
                printf("Enter data: ");
                scanf("%d", &data);
```

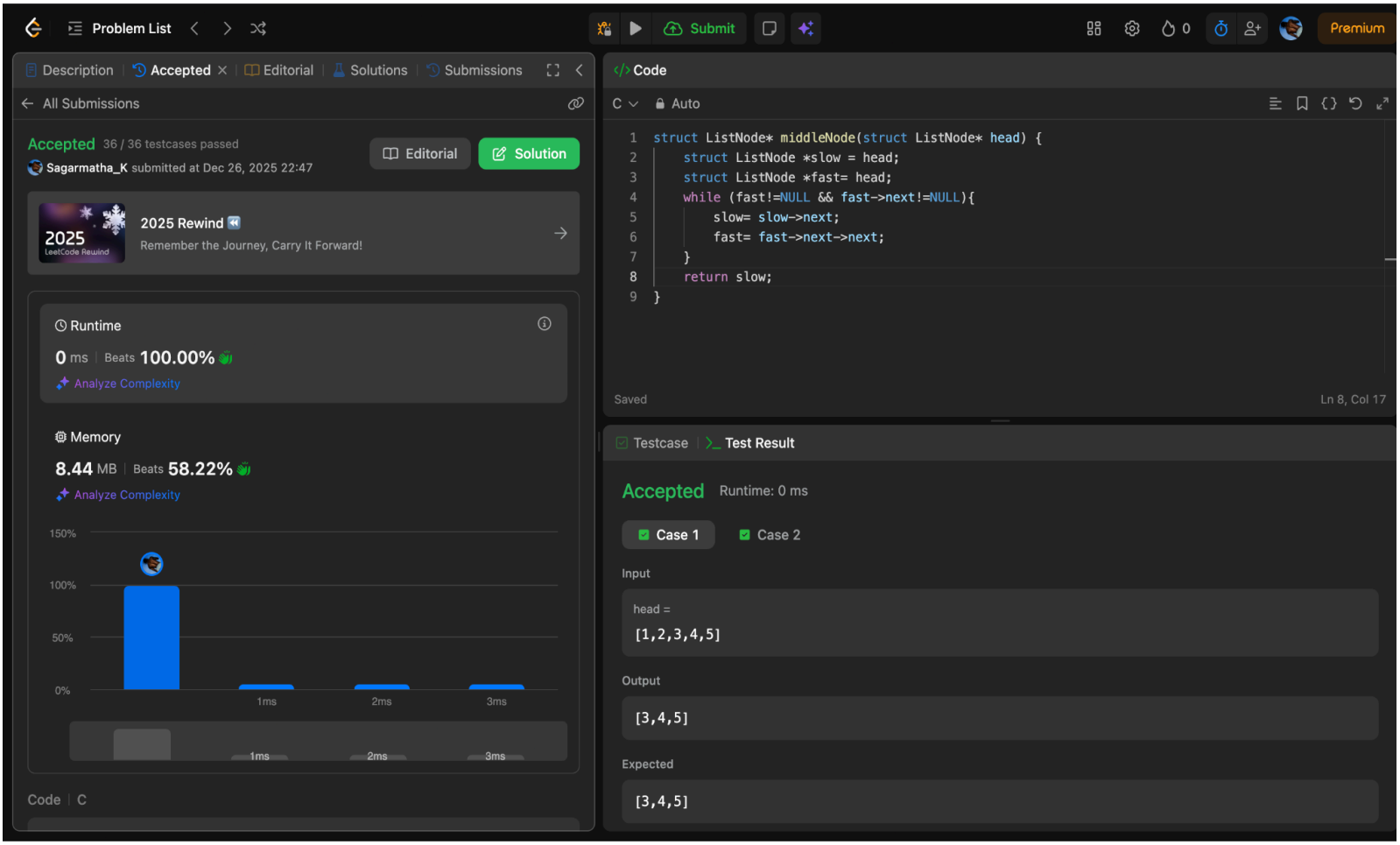
```
            insertAtEnd(data);
            break;
        case 3:
            printf("Enter data: ");
            scanf("%d", &data);
            printf("Enter pos: ");
            scanf("%d",&pos);
            insertAtPosition(data, pos);
            break;
        case 4:
            printf("Linked List: ");
            display();
            break;
        case 5:
            exit(0);
        default:
            printf("Invalid choice\n");
      }
    }
    return 0;
}
```

**OUTPUT:**



(b) LEETCODE-2:

**You are given the head of a non-empty singly linked list. Your task is to find and return the “middle” node of this linked list. If the list has an odd number of nodes, return the exact middle node. If the list has an even number of nodes, there will be two middle nodes, and you should return the second one. The returned node should itself be a linked list starting from that middle node onward.**



# LAB program-5

**WAP to Implement Singly Linked List with following operations**
**a) Create a linked list.**
**b) Deletion of first element, specified element and last element in the list.**
**c) Display the contents of the linked list.**

//CODE:

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node*next;
};


struct Node *head= NULL;


void createList(int n){
    struct Node *newNode, *temp;
    int data, i;

    if (n<=0){
        printf("NUmber of nodes should be greater then 0.\n");
        return;}

    for (i=1; i<=n;i++){
        newNode= (struct Node*)malloc(sizeof(struct Node));
        if (newNode== NULL){
            printf("Memory allocation failed. \n");
            return;
        }
        printf("Enter the data to be inserted:\n");
        scanf("%d", &data);
        newNode->data=data;
        newNode->next=NULL;
```

```
        if (head== NULL){
            head= newNode;
        }else{
            temp->next= newNode;
        }
        temp= newNode;
    }
    printf("\nLinked List created Successfully.\n");
}

void deletAtBegining(){
    struct Node *temp;
    temp=head;
    head=head->next;
    printf("deleted element is %d",temp->data);
    free(temp);
}

void deleteAtEnd(){
    struct Node *temp;
    struct Node *prev;
    temp=head;
    while (temp->next!=NULL)
    {
        prev=temp;
        temp=temp->next;
    }
    prev->next=NULL;
    printf("deleted element is %d",temp->data);
    free(temp);
}

void deleteAtAnyPos(int pos){
    int i;
    struct Node *temp;
    struct Node *prev;
    temp=head;
    if (pos<1){
        printf("invalid position.\n");
        return;
```

```
    }
    for (i=1; i<pos;i++){
        temp= temp->next;
    }
    prev->next= temp->next;
    printf("deleted element is %d",temp->data);
    free(temp);
}


void displayList(){
    struct Node *temp;
    temp=head;
    printf("the elements are:");
    while (temp!=NULL)
   {
    printf("%d",temp->data);
    temp= temp->next;
   }

}



int main(){
    int choice, n, data, pos;
    while (1){
        printf("\n----Singly Linked List Operations-------\n");
        printf("1.Create Linked List\n");
        printf("2. Delete at the Begining\n");
        printf("3. Delete at any Postion\n");
        printf("4. Delete at the End\n");
        printf("5. Display List\n");
        printf("6. Exit\n");
        printf("Enter your Choice:");
        scanf("%d", &choice);

        switch (choice)
        {
        case 1:
```

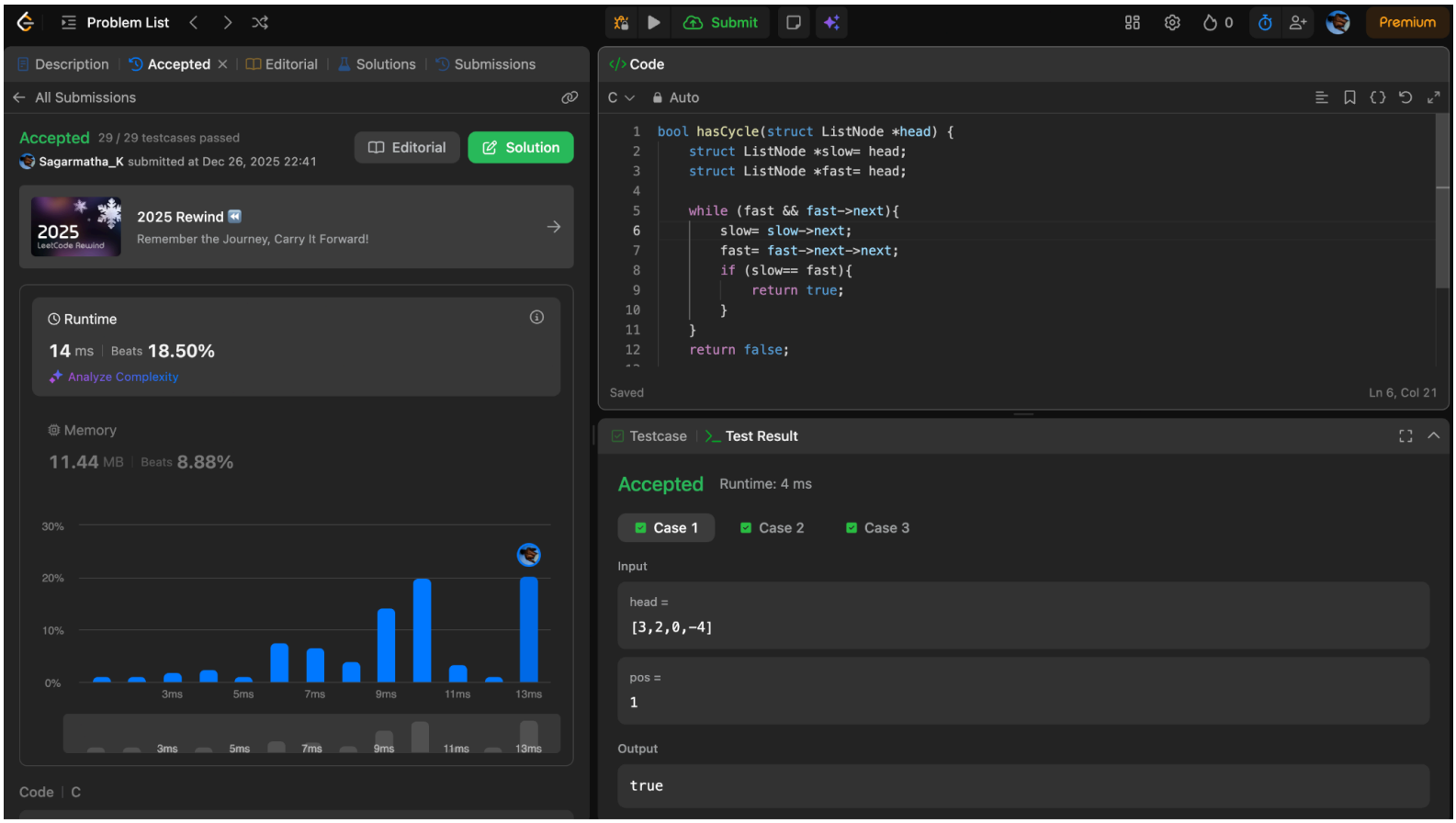
```
            printf("enter number of nodes:");
            scanf("%d",&n);
            createList(n);
            break;
        case 2:
            deletAtBegining();
            break;
        case 3:
            printf("enter the Postion:");
            scanf("%d",&data);
            deleteAtAnyPos(data);
            break;
        case 4:
            deleteAtEnd();
            break;
        case 5:
            displayList();
            break;
        case 6:
            printf("Exiting.....\n");
            exit(0);
        default:
            printf("Invalid choice. Try again. \n");
        }
    }
    return 0;
}
```

## OUTPUT:

```
sagarmathakhatri@Sagarmathas-MacBook-Air Coding % cd "/Users/sagarmathakhatri/Desktop/imp/Coding/" && gcc lab5.c -o lab5 && "/Users/sagarmathakhatri/Desktop/imp/Coding/"lab5

---- Singly Linked List Operations ----
1. Create Linked List
2. Delete at Beginning
3. Delete at Any Position
4. Delete at End
5. Display List
6. Exit
Enter your choice: 1
Enter number of nodes: 5
Enter data: 10
Enter data: 20
Enter data: 30
Enter data: 40
Enter data: 50
Linked List created successfully.

---- Singly Linked List Operations ----
1. Create Linked List
2. Delete at Beginning
3. Delete at Any Position
4. Delete at End
5. Display List
6. Exit
Enter your choice: 2
Deleted element is 10

---- Singly Linked List Operations ----
1. Create Linked List
2. Delete at Beginning
3. Delete at Any Position
4. Delete at End
5. Display List
6. Exit
Enter your choice: 3
Enter position: 3
Deleted element is 40

---- Singly Linked List Operations ----
1. Create Linked List
2. Delete at Beginning
3. Delete at Any Position
4. Delete at End
5. Display List
6. Exit
Enter your choice: 4
Deleted element is 50

---- Singly Linked List Operations ----
1. Create Linked List
2. Delete at Beginning
3. Delete at Any Position
4. Delete at End
5. Display List
6. Exit
Enter your choice: 5
The elements are: 20 -> 30 -> NULL
```

(b) LEETCODE-3:

**You are given the head of a singly linked list. The linked list may contain a cycle — that is, a node's `next` pointer might point to some previously visited node, forming a loop instead of terminating with `NULL`. Your task is to determine whether the linked list contains a cycle. Return `true` if there is a cycle in the list, and `false` if there is no cycle and the list ends normally. The challenge typically expects an efficient solution using constant extra space.**



# LAB program-6

**WAP to Implement Single Link List with following operations:**

1. **Sort the linked list,**
2. **Reverse the linked list,**
3. **Concatenation of two linked lists.**

//CODE:

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node*next;
};


struct Node *head1= NULL;
struct Node *head2= NULL;


void createList(int n){
    struct Node *newNode, *temp;
    int data, i;

    if (n<=0){
        printf("NUmber of nodes should be greater then 0.\n");
        return;}

    for (i=1; i<=n;i++){
        newNode= (struct Node*)malloc(sizeof(struct Node));
        if (newNode== NULL){
            printf("Memory allocation failed. \n");
            return;
        }
        printf("Enter the data to be inserted:\n");
        scanf("%d", &data);
        newNode->data=data;
```

```
        newNode->next=NULL;
        if (head1== NULL){
            head1= newNode;
        }else{
            temp->next= newNode;
        }
        temp= newNode;
    }
    printf("\nLinked List created Successfully.\n");
}

void createList2(int n){
    struct Node *newNode, *temp;
    int data, i;

    if (n<=0){
        printf("NUmber of nodes should be greater then 0.\n");
        return;}

    for (i=1; i<=n;i++){
        newNode= (struct Node*)malloc(sizeof(struct Node));
        if (newNode== NULL){
            printf("Memory allocation failed. \n");
            return;
        }
        printf("Enter the data to be inserted:\n");
        scanf("%d", &data);
        newNode->data=data;
        newNode->next=NULL;
        if (head1== NULL){
            head1= newNode;
        }else{
            temp->next= newNode;
        }
        temp= newNode;
    }
    printf("\nLinked List created Successfully.\n");
}
```

```
void sortlist(struct Node *head) {
    struct Node *i, *j;
    int tempData;

    if (head == NULL) return;  // handle empty list

    for (i = head; i->next != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->data > j->data) {
                // swap data
                tempData = i->data;
                i->data = j->data;
                j->data = tempData;
            }
        }
    }
}


void reverse(){
    struct Node* reverseList(struct Node *head){
        struct Node *prev=NULL, *curr= head, *next=NULL;
        while (curr!=NULL){
            next= curr->next;
            curr->next = prev;
            prev= curr;
            curr= next;
        }
        printf("great sucess \n");
    }
}

void concatenateion(){
    struct Node* concatenateion(struct Node *head1, struct Node *head2){
        struct Node *temp;
        if (head1==NULL) return head2;
        if (head2==NULL) return head1;

        temp= head1;
        while(temp->next !=NULL){
```

```
            temp= temp->next;
        }
        temp->next =head2;
        printf("linked list conatenated\n");
        return head1;

    }
}

void displayList(){
    struct Node *temp;
    temp=head1;
    printf("the elements are:");
    while (temp!=NULL)
   {
    printf("%d",temp->data);
    temp= temp->next;
   }
   struct Node *temp1;
    temp1=head2;
    printf("the elements are:");
    while (temp!=NULL)
   {
    printf("%d",temp1->data);
    temp= temp1->next;
   }

}



int main(){
    int choice, n, data, pos;
    while (1){
        printf("\n----Singly Linked List Operations-------\n");
        printf("1.Create Linked List\n");
        printf("2.Create second Linked List\n");
        printf("3. Sort the list\n");
        printf("4. Reverse the list\n");
        printf("5. Concatenate the list\n");
```

```
        printf("6. Display List\n");
        printf("7. Exit\n");
        printf("Enter your Choice:");
        scanf("%d", &choice);

        switch (choice)
        {
        case 1:
            printf("enter number of nodes:");
            scanf("%d",&n);
            createList(n);
            break;
        case 2:
            createList2(data);
            break;
        case 3:
            sortlist(data);
            break;
        case 4:
            printf("reversed first list:\n");
            reverse(head1);
            printf("reversed second list:\n");
            reverse(head2);
            break;
        case 5:
            concatenateion(head1, head2);
            break;
        case 6:
            displayList();
        case 7:
            printf("Exiting......\n");
            exit(0);
        default:
            printf("Invalid choice. Try again. \n");
        }
    }
    return 0;
}
```

## OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Enter data: 30
Enter data: 40

---- Singly Linked List Operations ----
1. Create First List
2. Create Second List
3. Sort Both Lists
4. Reverse Both Lists
5. Concatenate Lists
6. Display Lists
7. Exit
Enter choice: 2
Enter number of nodes: 2
Enter data: 99
Enter data: 88

---- Singly Linked List Operations ----
1. Create First List
2. Create Second List
3. Sort Both Lists
4. Reverse Both Lists
5. Concatenate Lists
6. Display Lists
7. Exit
Enter choice: 3
Lists sorted

---- Singly Linked List Operations ----
1. Create First List
2. Create Second List
3. Sort Both Lists
4. Reverse Both Lists
5. Concatenate Lists
6. Display Lists
7. Exit
Enter choice: 4
Lists reversed

---- Singly Linked List Operations ----
1. Create First List
2. Create Second List
3. Sort Both Lists
4. Reverse Both Lists
5. Concatenate Lists
6. Display Lists
7. Exit
Enter choice: 5
Lists concatenated

---- Singly Linked List Operations ----
1. Create First List
2. Create Second List
3. Sort Both Lists
4. Reverse Both Lists
5. Concatenate Lists
6. Display Lists
7. Exit
Enter choice: 6
First List: 40 -> 30 -> 20 -> 10 -> 99 -> 88 -> NULL
Second List: 99 -> 88 -> NULL
```

**(b) WAP to Implement Single Link List to simulate Stack & Queue Operations.**

//CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node* next;
};

struct node* push(struct node* top, int x){
    struct node* p = malloc(sizeof(struct node));
    p->data = x;
    p->next = top;
    return p;
}

struct node* pop(struct node* top){
    if(top){
        printf("Popped %d\n",top->data);
        struct node* t = top;
        top = top->next;
        free(t);
    } else {
        printf("Stack is empty\n");
    }
    return top;
}

struct node* enqueue(struct node* rear, struct node** front, int x){
    struct node* p = malloc(sizeof(struct node));
    p->data = x;
    p->next = NULL;

    if(*front == NULL) *front = p;
    else rear->next = p;
```

```
    printf("Enqueued %d\n", x);
    return p;
}

struct node* dequeue(struct node* front){
    if(front){
        printf("Dequeued %d\n",front->data);
        struct node* t = front;
        front = front->next;

        free(t);
    } else {
        printf("Queue is empty\n");
    }
    return front;
}

void display(struct node* head){
    if(head == NULL){
        printf("Empty\n");
        return;
    }
    while(head){
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main(){
    struct node *top = NULL, *front = NULL, *rear = NULL;
    int c, x;

    printf("\nMENU\n");
    printf("1. Stack - Push\n");
    printf("2. Stack - Pop\n");
    printf("3. Queue - Enqueue\n");
    printf("4. Queue - Dequeue\n");
    printf("5. Stack - Display\n");
    printf("6. Queue - Display\n");
```

```
printf("7. Exit\n");
printf("\n");

while(1){
    printf("\nEnter choice: ");
    scanf("%d", &c);

    switch(c){

        case 1:
            printf("Enter value to push: ");
            scanf("%d", &x);
            top = push(top, x);
            printf("Pushed %d\n", x);
            break;

        case 2:
            top = pop(top);
            break;

        case 3:
            printf("Enter value to enqueue: ");
            scanf("%d", &x);
            rear = enqueue(rear, &front, x);
            break;

        case 4:
            front = dequeue(front);
            if(front == NULL) rear = NULL;
            break;

        case 5:
            printf("Stack contents: ");
            display(top);
            break;

        case 6:
            printf("Queue contents: ");
            display(front);
            break;
```

```
            case 7:
                printf("Exiting program.\n");
                return 0;

            default:
                printf("Invalid choice. Try again.\n");
        }
    }
}
```

**OUTPUT:**

# LAB program-7

**WAP to Implement doubly link list with primitive operations**
**a) Create a doubly linked list.**
**b) Insert a new node to the left of the node.**
**c) Delete the node based on a specific value**
**d) Display the contents of the list**

//CODE:

```
#include <stdio.h>
#include <stdlib.h>

/* Node definition */
struct node {
    int data;
    struct node *prev;
    struct node *next;
};

/* Global pointers */
struct node *head = NULL;
struct node *tail = NULL;

/* Create doubly linked list */
void creation(int n) {
    struct node *newNode;
    int data, i;

    for (i = 1; i <= n; i++) {
        printf("Enter data for node %d: ", i);
        scanf("%d", &data);

        newNode = (struct node *)malloc(sizeof(struct node));
        newNode->data = data;
        newNode->prev = newNode->next = NULL;
```

```
        if (head == NULL) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
}

/* Insert at beginning */
void insert_at_beginning(int data) {
    struct node *newNode = (struct node *)malloc(sizeof(struct node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = head;

    if (head == NULL) {
        head = tail = newNode;
    } else {
        head->prev = newNode;
        head = newNode;
    }
}

/* Insert at end */
void insert_at_end(int data) {
    struct node *newNode = (struct node *)malloc(sizeof(struct node));
    newNode->data = data;
    newNode->next = NULL;

    if (tail == NULL) {
        head = tail = newNode;
        newNode->prev = NULL;
    } else {
        newNode->prev = tail;
        tail->next = newNode;
        tail = newNode;
    }
}
```

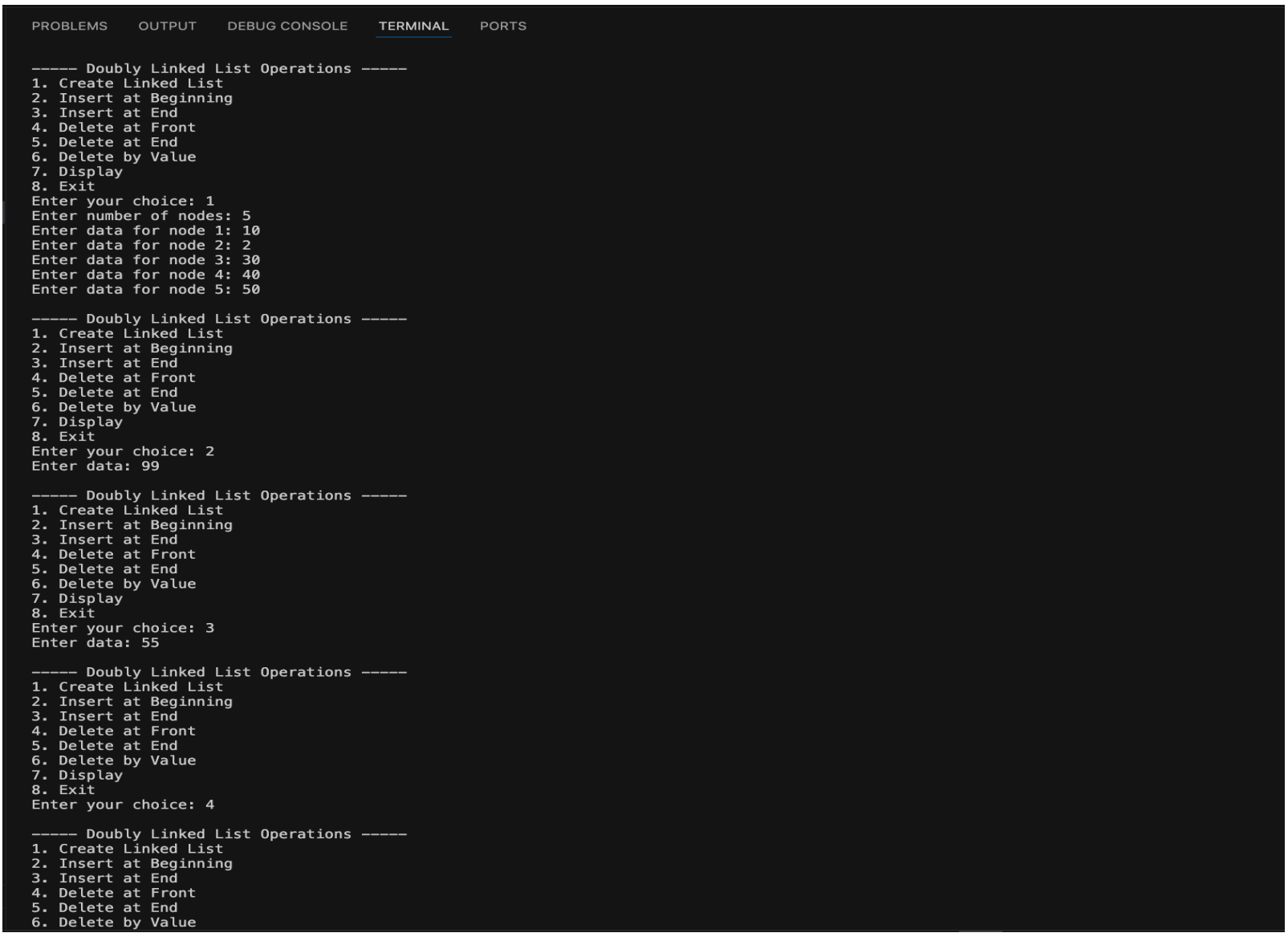
```
/* Delete at front */
void delete_at_front() {
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    temp = head;
    head = head->next;

    if (head != NULL)
        head->prev = NULL;
    else
        tail = NULL;

    free(temp);
}

/* Delete at end */
void delete_at_end() {
    struct node *temp;

    if (tail == NULL) {
        printf("List is empty\n");
        return;
    }

    temp = tail;
    tail = tail->prev;

    if (tail != NULL)
        tail->next = NULL;
    else
        head = NULL;

    free(temp);
}
```

```
/* Delete by value */
void delete_by_value(int value) {
    struct node *temp = head;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    while (temp != NULL && temp->data != value) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Value not found\n");
        return;
    }

    if (temp == head) {
        delete_at_front();
    } else if (temp == tail) {
        delete_at_end();
    } else {
        temp->prev->next = temp->next;
        temp->next->prev = temp->prev;
        free(temp);
    }
}

/* Display list */
void display() {
    struct node *temp = head;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Doubly Linked List: ");
```

```
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

/* Main menu */
int main() {
    int choice, n, data;

    while (1) {
        printf("\n----- Doubly Linked List Operations -----\n");
        printf("1. Create Linked List\n");
        printf("2. Insert at Beginning\n");
        printf("3. Insert at End\n");
        printf("4. Delete at Front\n");
        printf("5. Delete at End\n");
        printf("6. Delete by Value\n");
        printf("7. Display\n");
        printf("8. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
        case 1:
            printf("Enter number of nodes: ");
            scanf("%d", &n);
            creation(n);
            break;

        case 2:
            printf("Enter data: ");
            scanf("%d", &data);
            insert_at_beginning(data);
            break;

        case 3:
            printf("Enter data: ");
            scanf("%d", &data);
```

```
            insert_at_end(data);
            break;

        case 4:
            delete_at_front();
            break;

        case 5:
            delete_at_end();
            break;

        case 6:
            printf("Enter value to delete: ");
            scanf("%d", &data);
            delete_by_value(data);
            break;

        case 7:
            display();
            break;

        case 8:
            exit(0);

        default:
            printf("Invalid choice\n");
        }
    }
    return 0;
}
```

## OUTPUT:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
----- Doubly Linked List Operations -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Delete at Front
5. Delete at End
6. Delete by Value
7. Display
8. Exit
Enter your choice: 1
Enter number of nodes: 5
Enter data for node 1: 10
Enter data for node 2: 2
Enter data for node 3: 30
Enter data for node 4: 40
Enter data for node 5: 50
----- Doubly Linked List Operations -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Delete at Front
5. Delete at End
6. Delete by Value
7. Display
8. Exit
Enter your choice: 2
Enter data: 99
----- Doubly Linked List Operations -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Delete at Front
5. Delete at End
6. Delete by Value
7. Display
8. Exit
Enter your choice: 3
Enter data: 55
----- Doubly Linked List Operations -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Delete at Front
5. Delete at End
6. Delete by Value
7. Display
8. Exit
Enter your choice: 4
----- Doubly Linked List Operations -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Delete at Front
5. Delete at End
6. Delete by Value

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Enter data: 55

----- Doubly Linked List Operations -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Delete at Front
5. Delete at End
6. Delete by Value
7. Display
8. Exit
Enter your choice: 4

----- Doubly Linked List Operations -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Delete at Front
5. Delete at End
6. Delete by Value
7. Display
8. Exit
Enter your choice: 5

----- Doubly Linked List Operations -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Delete at Front
5. Delete at End
6. Delete by Value
7. Display
8. Exit
Enter your choice: 6
Enter value to delete: 99
Value not found

----- Doubly Linked List Operations -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Delete at Front
5. Delete at End
6. Delete by Value
7. Display
8. Exit
Enter your choice: 7
Doubly Linked List: 10 <-> 2 <-> 30 <-> 40 <-> 50 <-> NULL

----- Doubly Linked List Operations -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Delete at Front
5. Delete at End
6. Delete by Value
7. Display
8. Exit
Enter your choice: 8
sagarmathakhatri@Sagarmathas-MacBook-Air Coding %
```

# LAB program-8

**(a) Write a program:**

**(1) ToconstructabinarySearchtree.**

**(2) To traverse the tree using all the methods i.e., in-order, preorder and post order**

**(3) To display the elements in the tree.**

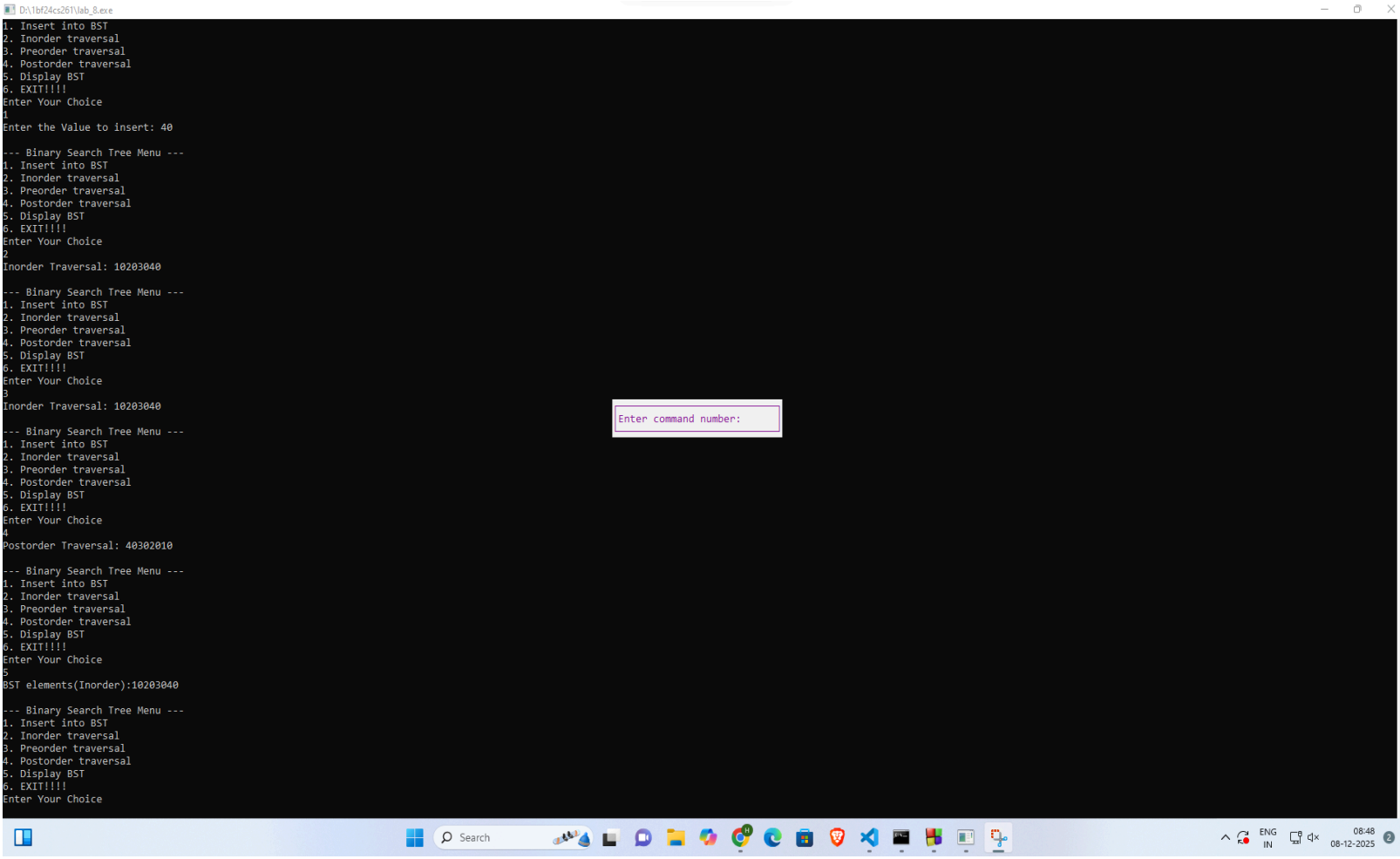
//CODE:

```
# include <stdio.h>
# include <stdlib.h>

struct Node{
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int value){
    struct Node *newNode= (struct Node*)malloc(sizeof(struct Node));
    newNode->data= value;
    newNode->left= newNode->right=NULL;
    return newNode;
}


struct Node* insert (struct Node *root, int value){
    if (root== NULL){
        return createNode(value);
    }

    if (value<root->data){
        root->left= insert(root->left, value);
    }else if(value> root->data){
        root->right= insert(root->right, value);
    }
    return root;
}
```

```
// traversals
void inorder(struct Node *root){
    if (root==NULL) return;
    inorder(root->left);
    printf("%d", root->data);
    inorder(root->right);
}

void preorder(struct Node *root){
    if (root==NULL) return;
    printf("%d", root->data);
    preorder(root->left);
    preorder(root->right);
}

void display(struct Node *root){
    printf("BST elements(Inorder):");
    inorder(root);
    printf("\n");
}


void postorder(struct Node *root){
    if (root==NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d", root->data);
}


int main(){
    struct Node *root= NULL;
    int choice, value;
    while (1){
        printf("\n--- Binary Search Tree Menu ---\n");
        printf("1. Insert into BST\n");
        printf("2. Inorder traversal\n");
        printf("3. Preorder traversal\n");
        printf("4. Postorder traversal\n");
        printf("5. Display BST\n");
```

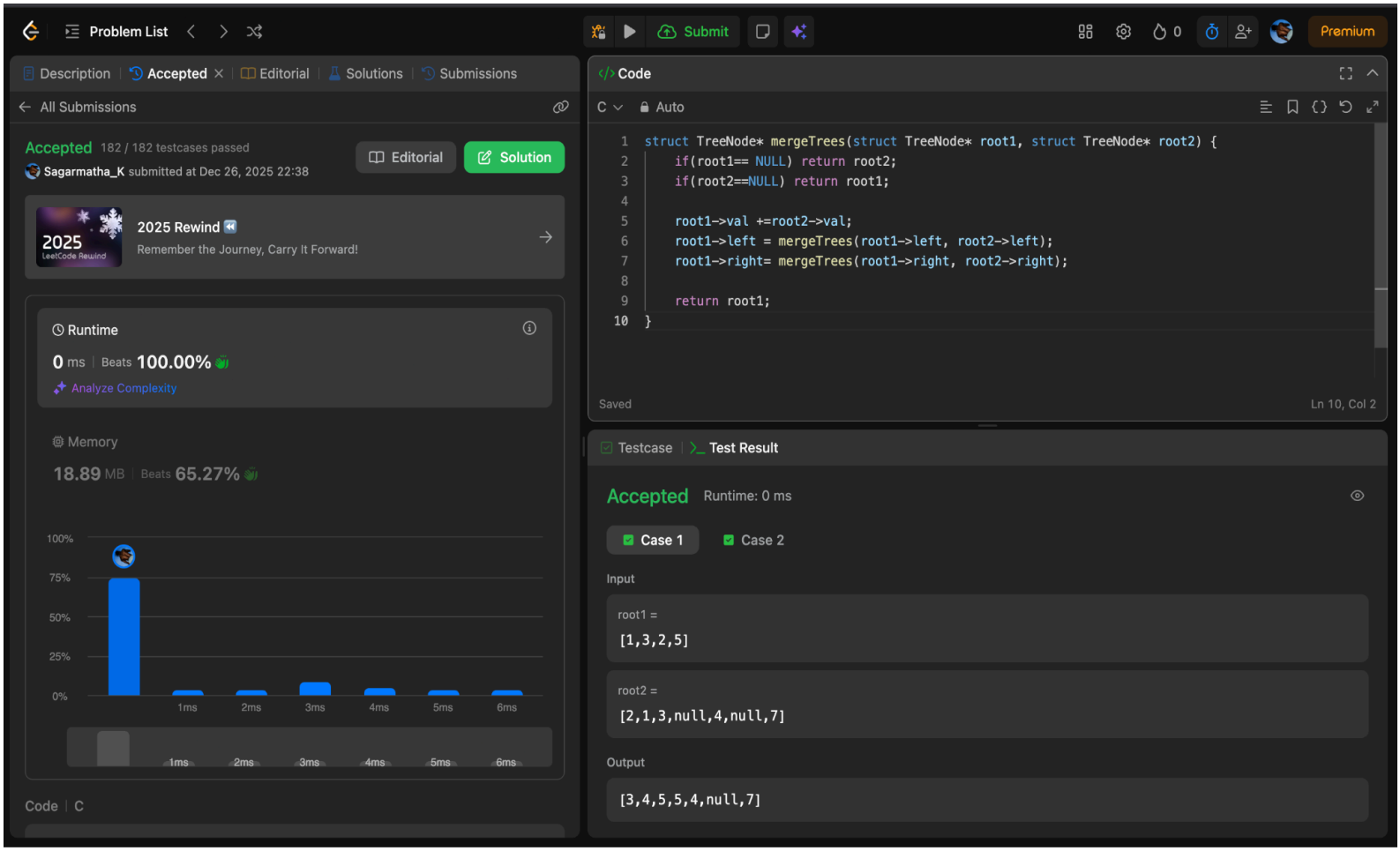
```
        printf("6. EXIT!!!!\n");
        printf("Enter Your Choice\n");
        scanf("%d",&choice);

        switch (choice)
        {
        case 1:
            printf("Enter the Value to insert: ");
            scanf("%d",&value);
            root= insert(root, value);
            break;
        case 2:
            printf("Inorder Traversal: ");
            inorder(root);
            printf("\n");
            break;
        case 3:
            printf("Inorder Traversal: ");
            preorder(root);
            printf("\n");
            break;
        case 4:
            printf("Postorder Traversal: ");
            postorder(root);
            printf("\n");
            break;
        case 5:
            display(root);
            break;
        case 6:
            exit(0);
        default:
            printf("Invalid choice!!! Try Ahain. \n");
            break;
        }
    }
    return 0;
}
```

# OUTPUT:


D:\1bf24cs261\lab_8.exe
1. Insert into BST
2. Inorder traversal
3. Preorder traversal
4. Postorder traversal
5. Display BST
6. EXIT!!!!
Enter Your Choice
1
Enter the Value to insert: 40
--- Binary Search Tree Menu ---
1. Insert into BST
2. Inorder traversal
3. Preorder traversal
4. Postorder traversal
5. Display BST
6. EXIT!!!!
Enter Your Choice
2
Inorder Traversal: 10203040
--- Binary Search Tree Menu ---
1. Insert into BST
2. Inorder traversal
3. Preorder traversal
4. Postorder traversal
5. Display BST
6. EXIT!!!!
Enter Your Choice
3
Inorder Traversal: 10203040
--- Binary Search Tree Menu ---
1. Insert into BST
2. Inorder traversal
3. Preorder traversal
4. Postorder traversal
5. Display BST
6. EXIT!!!!
Enter Your Choice
4
Postorder Traversal: 40302010
--- Binary Search Tree Menu ---
1. Insert into BST
2. Inorder traversal
3. Preorder traversal
4. Postorder traversal
5. Display BST
6. EXIT!!!!
Enter Your Choice
5
BST elements(Inorder):10203040
--- Binary Search Tree Menu ---
1. Insert into BST
2. Inorder traversal
3. Preorder traversal
4. Postorder traversal
5. Display BST
6. EXIT!!!!
Enter Your Choice
Enter command number:
Search
ENG
IN
08:48
08-12-2025

(b) LEETCODE-4

**You are given the roots of two binary trees, `root1` and `root2`. Your task is to merge these two trees into a new binary tree. The merge rule is that if two nodes overlap (both are non-null), then their values should be added up to become the value of the merged node. If only one node exists in a position (the other is null), then the non-null node should be used directly in the merged tree. The function should return the root of the merged binary tree.**



# LAB program-9

**(a) Write a program to traverse a graph using BFS method.**

//CODE:

```
#include <stdio.h>

int graph[20][20], visited[20], n;

void BFS(int start) {
    int queue[20], front = 0, rear = 0;

    visited[start] = 1;
    queue[rear++] = start;

    while (front < rear) {
        int node = queue[front++];
        printf("%d ", node);

        for (int i = 0; i < n; i++) {
            if (graph[node][i] == 1 && !visited[i]) {
                visited[i] = 1;
                queue[rear++] = i;
            }
        }
    }
}

int main() {
    int start;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);
```

```
    for (int i = 0; i < n; i++)
        visited[i] = 0;

    printf("Enter starting vertex: ");
    scanf("%d", &start);

    printf("BFS traversal: ");
    BFS(start);

    return 0;
}
```

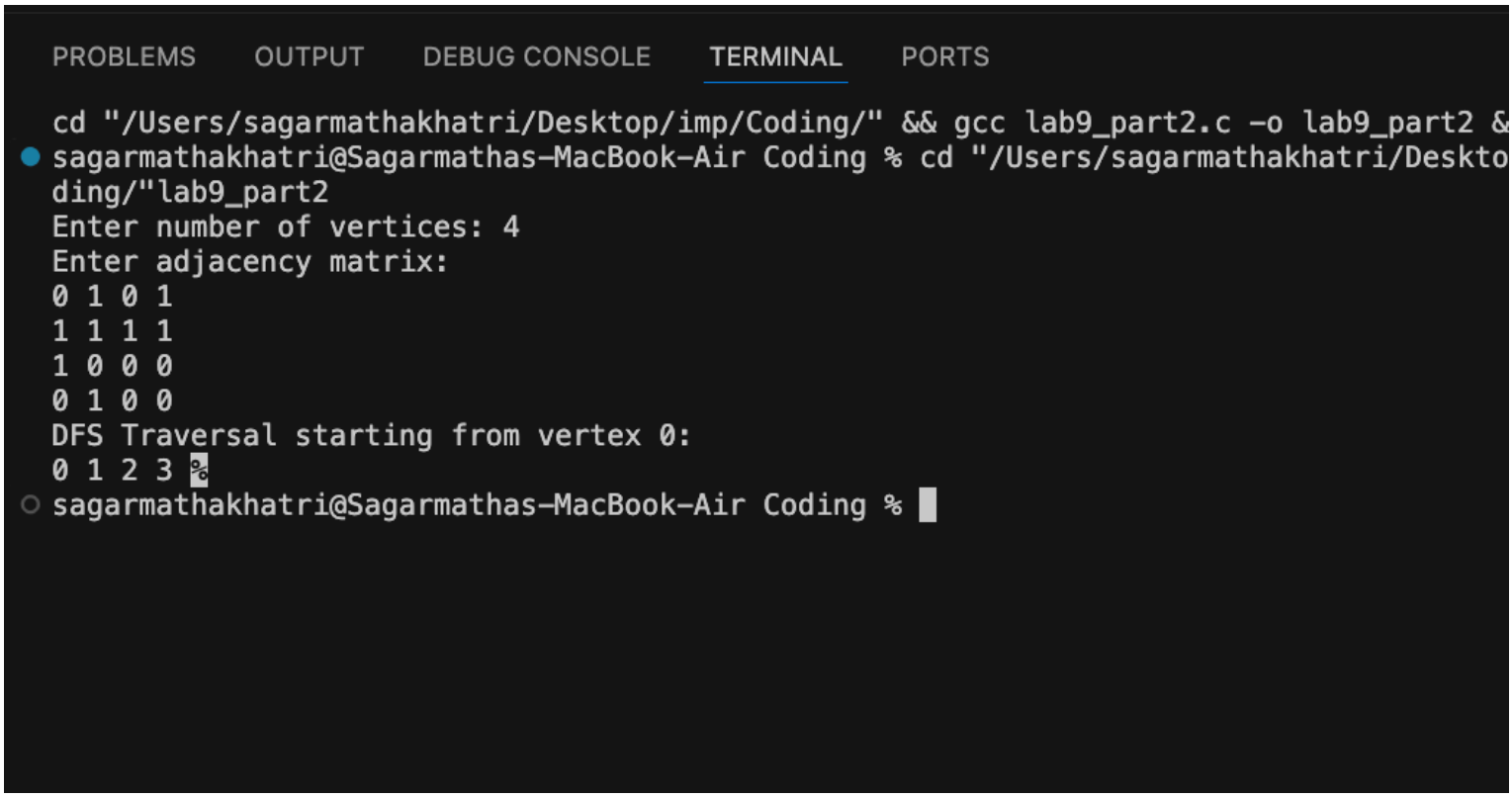
**OUTPUT**:

**(b) Write a program to check whether given graph is connected or not using DFS method.**

//CODE:

```
#include <stdio.h>
#define MAX 10
int visited[MAX];
int adj[MAX][MAX];
int n;
void DFS(int v) {
    visited[v] = 1;
    printf("%d ", v);

    for (int i = 0; i < n; i++) {
        if (adj[v][i] == 1 && !visited[i]) {
            DFS(i);
        }
    }
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
    for (int i = 0; i < n; i++)
        visited[i] = 0;
    printf("DFS Traversal starting from vertex 0:\n");
    DFS(0);

    return 0;
}
```

**OUTPUT:**


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
cd "/Users/sagarmathakhatri/Desktop/imp/Coding/" && gcc lab9_part2.c -o lab9_part2 &
sagarmathakhatri@Sagarmathas-MacBook-Air Coding % cd "/Users/sagarmathakhatri/Deskto
ding/"lab9_part2
Enter number of vertices: 4
Enter adjacency matrix:
0 1 0 1
1 1 1 1
1 0 0 0
0 1 0 0
DFS Traversal starting from vertex 0:
0 1 2 3 %
sagarmathakhatri@Sagarmathas-MacBook-Air Coding %

# LAB program-10

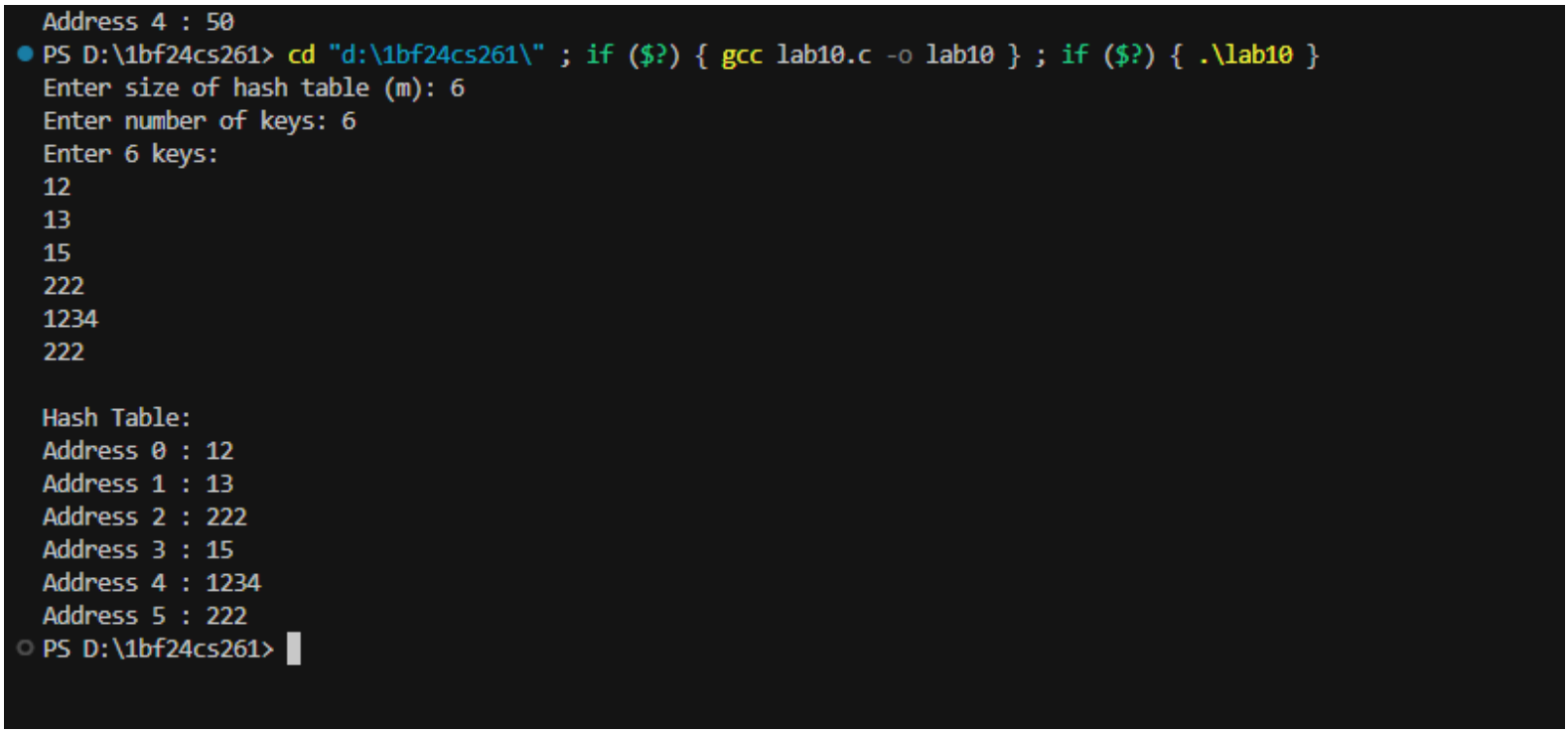
**Given a File of N employee records with a set K of Keys(4-digit) which uniquely determine the records in file F. Assume that file F is maintained in memory by a Hash Table (HT) of m memory locations with L as the set of memory addresses (2-digit) of locations in HT. Let the keys in K and addresses in L are integers. Design and develop a Program in C that uses Hash function H: K -> L as H(K)=K mod m (remainder method), and implement hashing technique to map a given key K to the address space L. Resolve the collision (if any) using linear probing.**

//CODE:

```
#include <stdio.h>
#define MAX 20

int hashTable[MAX];
int m;

void insert(int key)
{
    int index = key % m;

    if (hashTable[index] == -1)
    {
        hashTable[index] = key;
    }
    else
    {
        int i = 1;
        while (hashTable[(index + i) % m] != -1)
        {
            i++;
        }
        hashTable[(index + i) % m] = key;
    }
}
```

```
void display()
{
    printf("\nHash Table:\n");
    for (int i = 0; i < m; i++)
    {
        if (hashTable[i] != -1)
            printf("Address %d : %d\n", i, hashTable[i]);
        else
            printf("Address %d : Empty\n", i);
    }
}

int main()
{
    int n, key;

    printf("Enter size of hash table (m): ");
    scanf("%d", &m);

    printf("Enter number of keys: ");
    scanf("%d", &n);


    for (int i = 0; i < m; i++)
        hashTable[i] = -1;

    printf("Enter %d keys:\n", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &key);
        insert(key);
    }

    display();

    return 0;
}
```

**OUTPUT:**


Address 4 : 50
PS D:\1bf24cs261> cd "d:\1bf24cs261\" ; if ($?) { gcc lab10.c -o lab10 } ; if ($?) { .\lab10 }
Enter size of hash table (m): 6
Enter number of keys: 6
Enter 6 keys:
12
13
15
222
1234
222
Hash Table:
Address 0 : 12
Address 1 : 13
Address 2 : 222
Address 3 : 15
Address 4 : 1234
Address 5 : 222
PS D:\1bf24cs261>